SOFTWARE ENGINEERING & PROJECT MANAGEMENT
CIA-III

Volcanic Image/Data Observatory (VIDO)

A Lightweight Scientific Software System for Heterogeneous Terrestrial & Planetary Volcanology Exploration

By Tanisha Chhetri(2462160), Shomik Sahu(2462191), Paul Jestin Abhishek(2462192)

Software Engineering & Project Management (SEPM) • Final System Evaluation & Audit
Application Domain: Terrestrial (Earth WGS84) & Planetary Volcanology (Mars, Io, Venus)
Implementation Verification: Fully Verified Core MVP • 37 Passing Automated Tests

1. Problem Definition: Heterogeneous Scientific Data Management

CORE ENGINEERING QUESTION

“How can heterogeneous scientific observations be represented, validated, related, and explored in a coherent and maintainable software system?”

- **Heterogeneous Scientific Data:** Multispectral imagery, thermal radiometry, orbital geometry, and point coordinates originating from diverse instruments.
- **Application Domain Context:** Scientific observation datasets across Earth, Mars, Io, and Venus introducing non-standard spatial and temporal constraints.
- **Software Engineering Framing:** Volcanology is the application domain; software design, architecture, data modeling, and verification are the primary subject.

PRIMARY SOFTWARE ENGINEERING CHALLENGES

- **Coordinate Frame Disparities:** Standard Earth WGS84 (-180° to +180°) vs. Planetary IAU (0° to 360° Positive East) conventions.
- **Schema Rigidity & Bloat:** Forcing multi-sensor attributes into flat relational SQL schemas causes sparse, brittle tables with excessive NULL columns.
- **Observation / Event Conflation:** Conflating observational snapshot evidence with physical eruptive events degrades domain modeling integrity.
- **System Maintainability:** Delivering strict domain validation and clean data access without heavy, complex framework dependencies.

2. Requirements Engineering: Functional & Non-Functional Specifications

FUNCTIONAL REQUIREMENTS (FR)

- **FR-01 Observation Management:** Store and manage observation records across celestial bodies.
- **FR-02 Multi-Criteria Retrieval:** Query observations by date range, facet category, sensor source, and system.
- **FR-03 Event Association:** Relate observations to physical eruptive events using explicit relationship tags.
- **FR-04 Spatial Exploration:** Plot spatial markers on Earth Leaflet maps or 0°-360° planetary grids with fallback rules.
- **FR-05 Timeline Exploration:** Render synchronized dual-lane chronological timeline with ongoing event support.
- **FR-06 Domain Validation:** Enforce metadata facet validation and coordinate boundary rules prior to persistence.

NON-FUNCTIONAL REQUIREMENTS (NFR TARGETS)

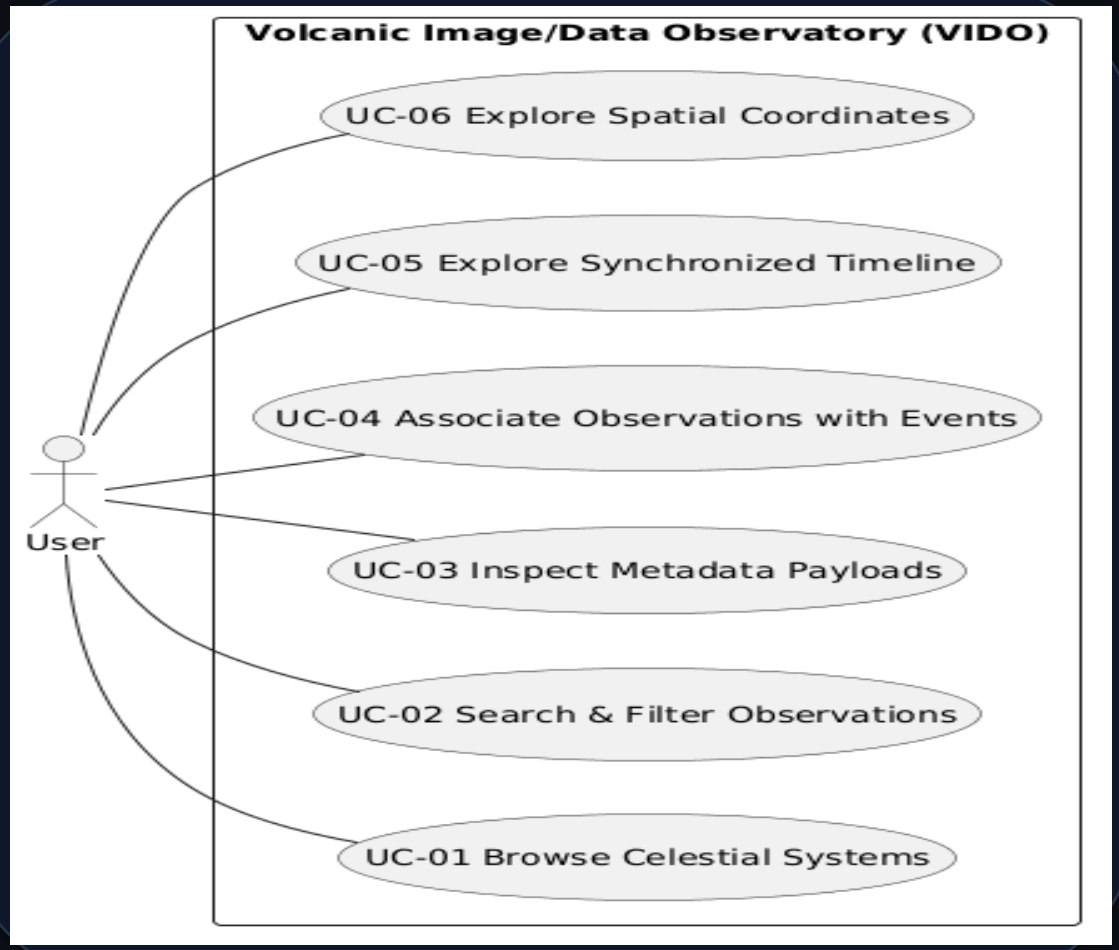
- **NFR-01 Maintainability:** Layered 5-tier architecture enforcing separation of concerns across presentation, API, services, repos, and DB.
- **NFR-02 Extensibility:** Modular facet schemas allowing new observation types without database schema migrations.
- **NFR-03 Testability:** Decoupled domain service layer enabling isolated automated testing (core domain logic test suite).
- **NFR-04 Data Integrity:** Schema boundary validation (Pydantic) and database relational transaction safety.
- **NFR-05 Performance & Modularity:** Lightweight architecture with zero heavy build tool overhead and low query latencies.

3. Use Case Analysis & Mandatory UML Use Case Diagram

USE CASE MODEL & USER INTERACTIONS

"The use-case model translates functional requirements into observable interactions between the user and VIDO."

- **UC-01 Browse Celestial Systems:** Select target body & explore system profiles.
- **UC-02 Search & Filter Observations:** Query by date, facet category, or source.
- **UC-03 Inspect Metadata Payloads:** View validated composite metadata facets.
- **UC-04 Associate Observations with Events:** Link evidence snapshots to eruptive events.
- **UC-05 Explore Synchronized Timeline:** Inspect dual-lane chronological feed.
- **UC-06 Explore Spatial Coordinates:** Render markers with spatial fallback rules.



4. Requirements Traceability: FR → UC → API → Service → Test

END-TO-END SYSTEM TRACEABILITY PIPELINE

Requirement (FR) → Use Case (UC) → API Endpoint → Service Layer → Repository / DB → Pytest Verification

FR-01 Observation Management | Use Case: UC-02 / UC-03 | API: POST /api/v1/observations

Service: ValidationService • Repo: ObservationRepository • Verification: test_models.py, test_api.py

FR-02 Multi-Criteria Retrieval | Use Case: UC-02 | API: GET /api/v1/observations

Service: ValidationService • Repo: ObservationRepository • Verification: test_api.py

FR-03 Event Association | Use Case: UC-04 | API: POST /api/v1/observation-event-links

Service: EventService • Repo: ObservationEventLinkRepository • Verification: test_domain_rules.py, test_services.py

FR-04 Spatial Exploration | Use Case: UC-06 | API: GET /api/v1/systems/{id}/spatial

Service: SpatialService / CoordinateService • Repo: VolcanicSystemRepository / ObservationRepo • Verification: test_domain_rules.py, test_repositories.py

FR-05 Timeline Exploration | Use Case: UC-05 | API: GET /api/v1/systems/{id}/timeline

Service: TimelineService • Repo: VolcanicEventRepository / ObservationRepo • Verification: test_services.py, test_api.py

FR-06 Domain Validation | Use Case: UC-02 / UC-03 | API: Pydantic API Schemas

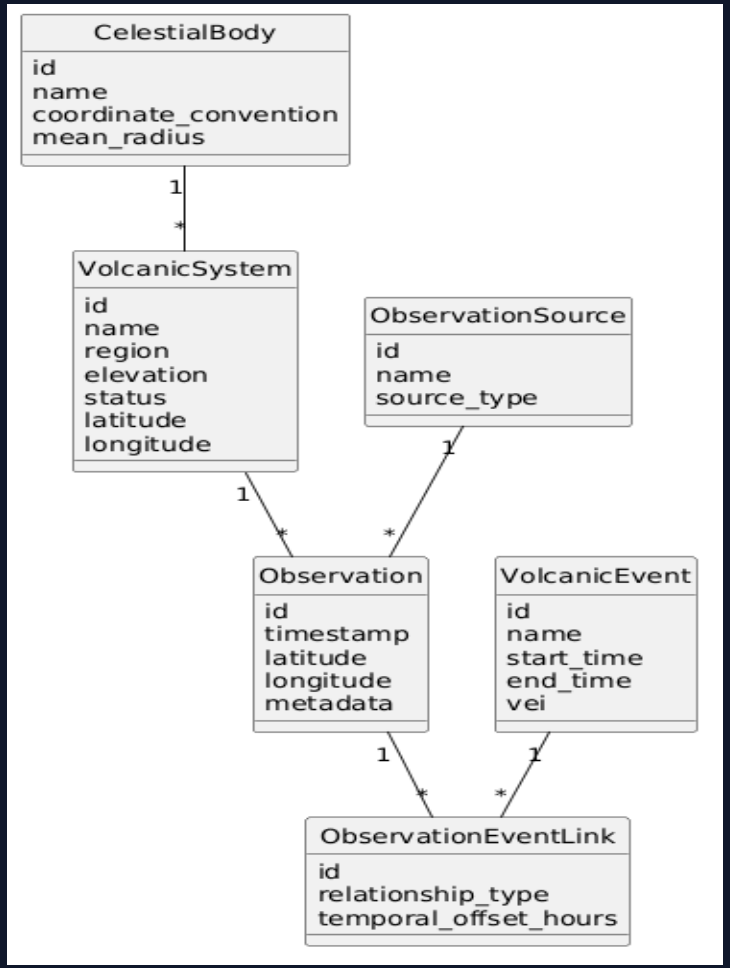
Service: ValidationService / CoordinateService • Persistence: Blocked until validation succeeds• Verification: test_models.py, test_domain_rules.py

5. Software Design & Mandatory UML Class Diagram

DOMAIN CLASS MODEL & RELATIONSHIPS

"Separating observations from physical events preserves distinct lifecycles and attributes while ObservationEventLink provides explicit association."

- **CelestialBody (1 → * VolcanicSystem):** Planet/Moon container establishing coordinate convention.
- **VolcanicSystem (1 → * Observation):** Volcanic feature defining geographic base coordinates.
- **ObservationSource (1 → * Observation):** Sensor platform or operating scientific agency.
- **Observation (1 → * Link):** Point-in-time observational evidence snapshot with metadata.
- **VolcanicEvent (1 → * Link):** Physical eruptive episode with duration (start_time, end_time).
- **ObservationEventLink:** Explicit junction entity connecting observations to events.



6. Composite Metadata Facet Architecture

MODULAR FACET COMPOSITION PATTERN

- **Observation Facet Structure:** Observation entity encapsulates non-mutually-exclusive metadata facets rather than rigid subclassing.
- **IMAGE Facet:** Spectral band, spatial resolution (m), cloud cover %, file format, sun elevation angle.
- **THERMAL Facet:** Brightness temperature (K), ambient temp (K), thermal flux (MW), anomaly flag.
- **PLANETARY_ORBITAL Facet:** Spacecraft altitude (km), solar incidence angle, emission angle, target planetary datum.
- **Validation Guarantee:** Pydantic v2 schemas reject malformed or out-of-bounds payloads at API boundaries.

REPRESENTATIVE COMPOSITE JSON PAYLOAD


```
{
  "active_facets": [
    "IMAGE", "THERMAL", "PLANETARY_ORBITAL"
  ],
  "image_metadata": {
    "spectral_band": "NEAR_INFRARED",
    "spatial_resolution_m": 10.0,
    "cloud_cover_percentage": 0.0
  },
  "thermal_metadata": {
    "brightness_temperature_kelvin": 245.5,
    "thermal_flux_mw": 12.4,
    "anomaly_flag": false
  },
  "orbital_metadata": {
    "spacecraft_altitude_km": 310.5,
    "solar_incidence_angle_deg": 62.1,
    "target_planetary_datum": "IAU_MARS_2000"
  }
}
```

7. System Architecture: 5-Layer Software Pipeline

Presentation Layer Vanilla HTML5 / CSS3 / ES6 JS	Renders responsive web UI, Leaflet Earth maps, Canvas 2D planetary grids, and interactive timeline visualizers without heavy frontend build tool overhead.
API Layer Python FastAPI REST Server	Exposes REST endpoints (/api/v1), serializes request/response payloads, executes Pydantic schema boundary validation, handles HTTP status routing, and serves Swagger OpenAPI docs.
Service Layer — Business & Domain Logic Domain & Business Services	Executes domain/business validation, coordinate resolution, spatial fallback rules, event rules, and timeline aggregation.
Repository Layer Data Access Abstraction	Encapsulates database SQL operations, providing clean, decoupled interfaces for systems, observations, events, and association links.
Database Layer SQLite 3 Relational Engine	Provides persistent, ACID-compliant storage with SQLite 3 JSON1 support for composite observation metadata facets.

8. Design Decisions & Engineering Trade-offs

Relational Core + JSON Metadata

Combines SQL relational structure with SQLite JSON1 metadata fields. Prevents schema rigidity while retaining query capabilities.

Decoupled Obs & Event Entities

Models observations (evidence snapshots) separately from events (phenomena). Prevents data loss when linking multi-event observations.

Body-Specific Coordinate Handling

Enforces Earth WGS84 (-180°..+180°) vs. Planetary IAU (0°..360° Positive East) conventions. Prevents spatial projection errors.

Explicit Spatial Fallback Rules

Applies a 3-step fallback pipeline (Obs Coords → Volcano Coords → UNLOCATED). Prevents coordinate fabrication.

Service / Repository Abstraction

Decouples business logic from persistence logic. Prevents SQL leaks into API controllers and enables unit test isolation.

Conservative MVP Scope Boundary

Focuses on robust core domain modeling, API, and validation rather than unverified complex modules. Prevents scope creep.

9. Validation & Business Rules Architecture

TIER 1: API SCHEMA VALIDATION (PYDANTIC)

- **Boundary Schema Enforcement:** Validates request JSON payloads at API entry points before database invocation.
- **Attribute Type Checking:** Guarantees correct string, float, integer, and datetime field formatting.
- **Facet Payload Validation:** Validates nested image_metadata, thermal_metadata, and orbital_metadata objects.
- **Value Range Rules:** Rejects negative Kelvin temperatures, out-of-range sensor angles, or invalid file extensions.

TIER 2: DOMAIN & BUSINESS RULE VALIDATION

→ **Body Coordinate Boundary Checks:** Enforces Earth WGS84 (-180°..+180°) vs. Planetary IAU (0°..360° Positive East) bounds.

→ **Temporal Event Sequence Rules:** Ensures event end_time cannot precede start_time (when end_time is provided).

→ **Relationship Classification Rules:** Enforces valid Observation ↔ Event tags (PRE_ERUPTIVE, CO_ERUPTIVE, POST_ERUPTIVE, UNRELATED).

→ **Spatial Fallback Resolution:** Classifies coordinates into OBSERVATION, VOLCANO_FALLBACK, or UNLOCATED states cleanly.

10. Development Process: Phased SDLC Workflow

1. Problem Definition & SRS

Formalized engineering problem statement and compiled Software Requirements Specification.

2. Requirements Audit

Audited domain parameters and mapped functional vs. non-functional requirement targets.

3. Data Layer & Entities

Designed SQLite relational tables, JSON metadata fields, and Pydantic validation schemas.

4. Business Logic Services

Implemented coordinate resolution, spatial fallback rules, event rules, and timeline aggregation.

5. API Layer

Exposed REST endpoints (/api/v1) with OpenAPI Swagger docs and robust HTTP error routing.

6. Frontend Interface

Built responsive web UI with Leaflet maps, Canvas 2D planetary grid, and timeline visualizers.

7. Automated Verification

Developed pytest test suite covering API routes, domain rules, schemas, and repositories.

8. Presentation & QC

Audited implementation claims against codebase evidence and prepared evaluation materials.

11. Scope Control & Risk Management Strategy

DELIBERATE SCOPE CONTROL

- **Implemented Core MVP Scope:** Unified domain model, multi-body CRS, composite facet validation, 5-layer pipeline, dual-lane timeline, spatial fallback visualizer, REST API, 37 automated tests.
- **Explicit Non-Implemented Future Scope:** Streaming satellite telemetry pipelines, computer vision thermal anomaly workers, GeoServer WMS integration, user authentication/authorization.
- **Scope Boundary Principle:** Maintaining strict boundaries prevented unfinished features from degrading core software quality.

RISK MITIGATION MATRIX

⚙ **Heterogeneous Data Distortion Risk:** Mitigated by Composite Metadata Facet validation pattern.

⚙ **Coordinate Frame Mismatch Risk:** Mitigated by CelestialBody explicit CRS conventions (WGS84 vs. IAU).

⚙ **Observation/Event Ambiguity Risk:** Mitigated by explicit ObservationEventLink junction modeling.

⚙ **Scope Creep Risk:** Mitigated by strict SRS specification and phase verification.

⚙ **Regression & Code Fragility Risk:** Mitigated by 37 passing automated pytest unit/integration tests.

12. Testing & System Verification Results

✓ 37 / 37 AUTOMATED TESTS PASSED

Fully verified core MVP implementation across API endpoints, domain rules, metadata schemas, repositories, and services.

API Endpoint Integration (15 tests)

API integration tests covering catalog, observation, event, spatial, and timeline REST endpoints.

Domain Rules & Boundaries (5 tests)

Validates coordinate boundary enforcement (Earth vs. Mars/Io) and temporal sequence validity.

Model & Facet Schemas (6 tests)

Tests Pydantic metadata facet validation (IMAGE, THERMAL, PLANETARY_ORBITAL) rejecting illegal payloads.

Repository & Persistence (5 tests)

Verifies SQLite 3 SQL/JSON data access, insertion transactions, and complex query filtering.

Business Service Logic (5 tests)

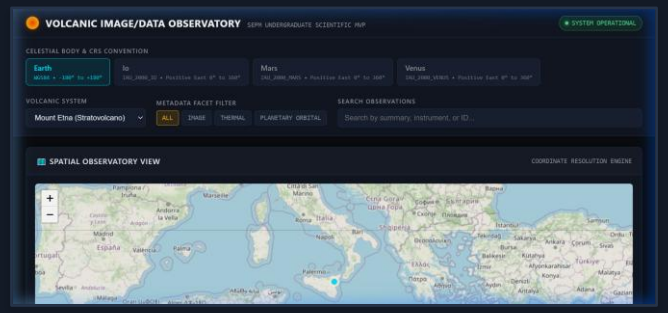
Verifies timeline aggregation logic, ongoing event handling (NULL end_time), and spatial fallback resolution.

Seed Data Verification (1 test)

Ensures initial seed dataset across Earth, Mars, Io, and Venus populates without relational errors.

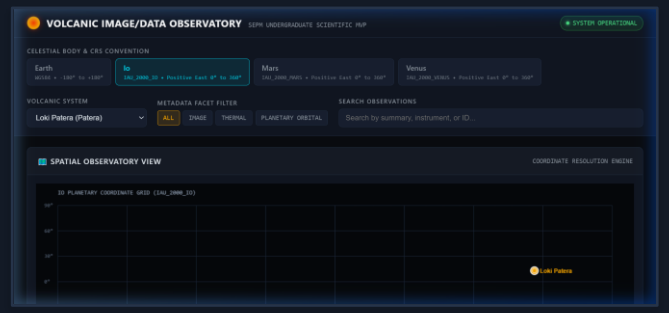
13. Implementation Demonstration: Software Evidence

Mount Etna / Earth



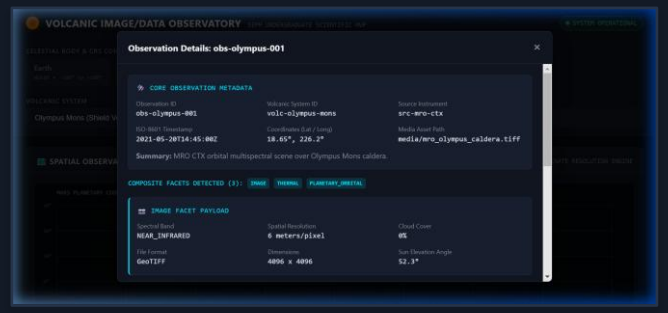
Leaflet Earth Map view rendering geographic spatial representation & eruptive timeline feed.

Loki Patera / Io



Canvas 2D Planetary Grid plotting Io thermal observations in 0°-360° Positive East CRS.

Olympus Mons / Mars



Composite Metadata Inspector modal rendering active IMAGE, THERMAL & PLANETARY_ORBITAL facets.

14. Team Contributions, Limitations & SEPM Conclusion

TEAM CONTRIBUTIONS & LIMITATIONS

- **Technical Implementation:** Architecture, database schema, FastAPI backend, business validation services, frontend UI, and pytest test suite.
- **Documentation & SRS:** Software Requirements Specification (SRS), domain requirements audit, system documentation, and architecture specifications.
- **Presentation & QC Audit:** Presentation deck, visual alignment, speaker notes, and quality audit.
- **Current Scope Limitations:** Manual dataset seeding; non-automated satellite ingestion; local SQLite storage rather than distributed database.

SEPM CONCLUSION

"VIDO demonstrates the design and implementation of a modular scientific information system capable of representing heterogeneous observations, enforcing domain rules, relating observations to events, exposing functionality through a layered API, and supporting interactive exploration with automated verification."

✓ **Layered Architecture:** Layered pipeline enforces separation of concerns and supports data integrity and testability.

✓ **Automated Verification:** 37 / 37 automated tests verify core domain rules, APIs, and repositories.

✓ **Future Scope Roadmap:** Telemetry streaming pipelines, CV anomaly detection workers, GeoServer WMS integration, and user authorization.